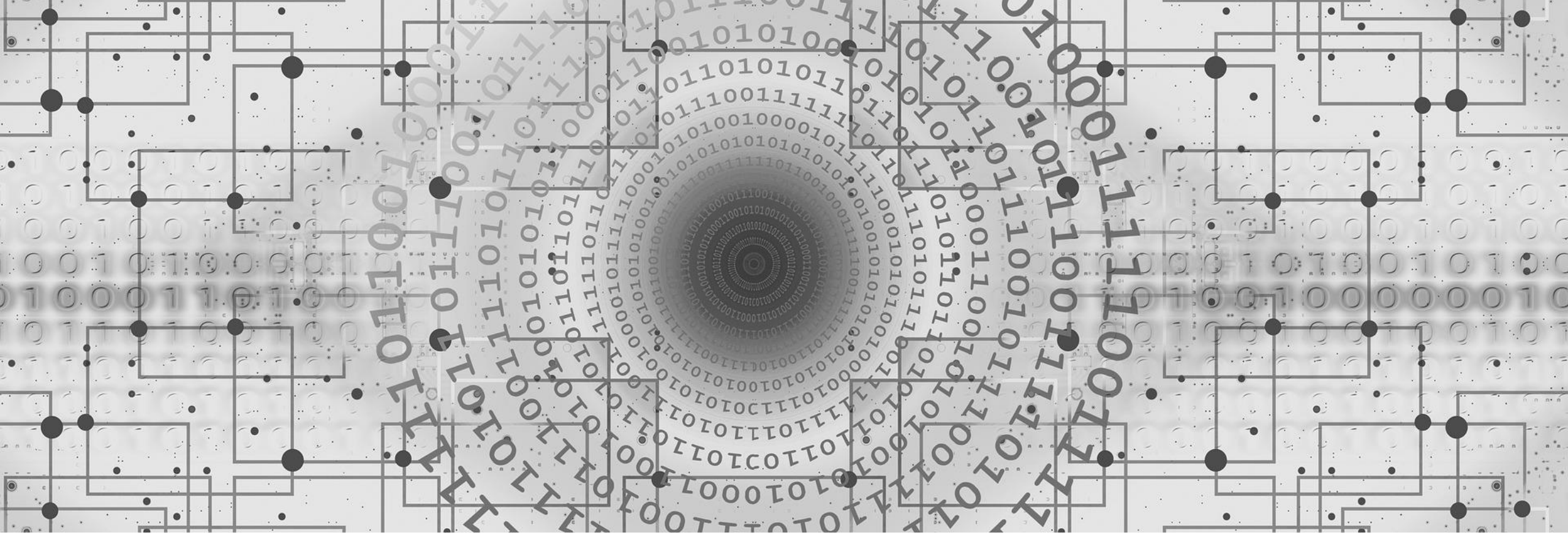


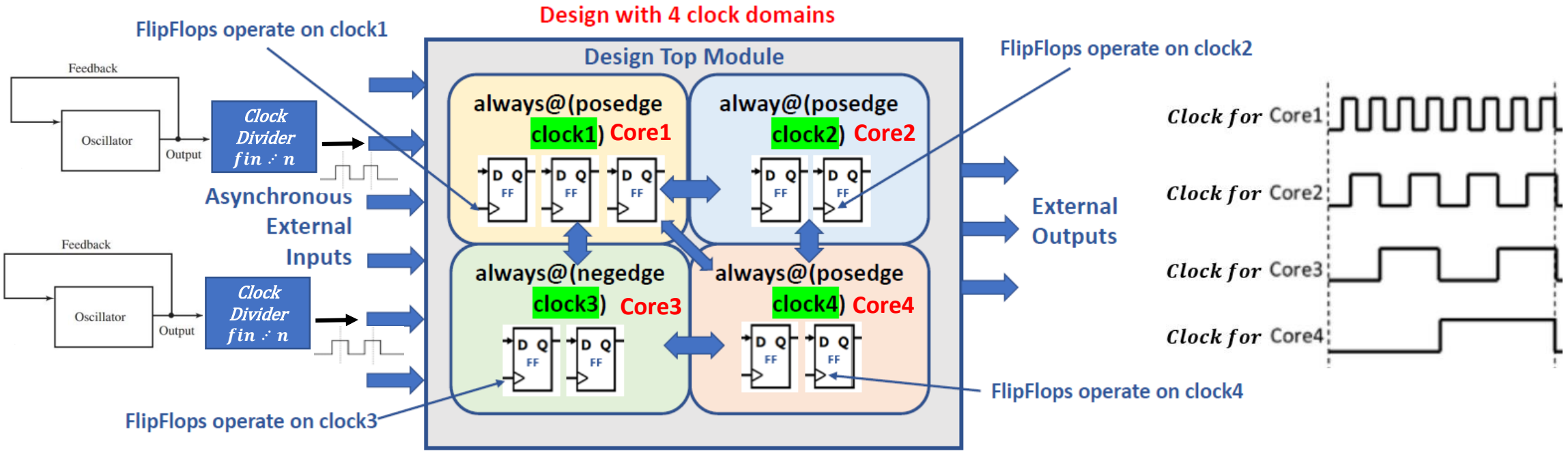
Frequency Divider (Clock Divider)

ECE-111 Advanced Digital Design Project

UC San Diego
JACOBS SCHOOL OF ENGINEERING
Electrical and Computer Engineering

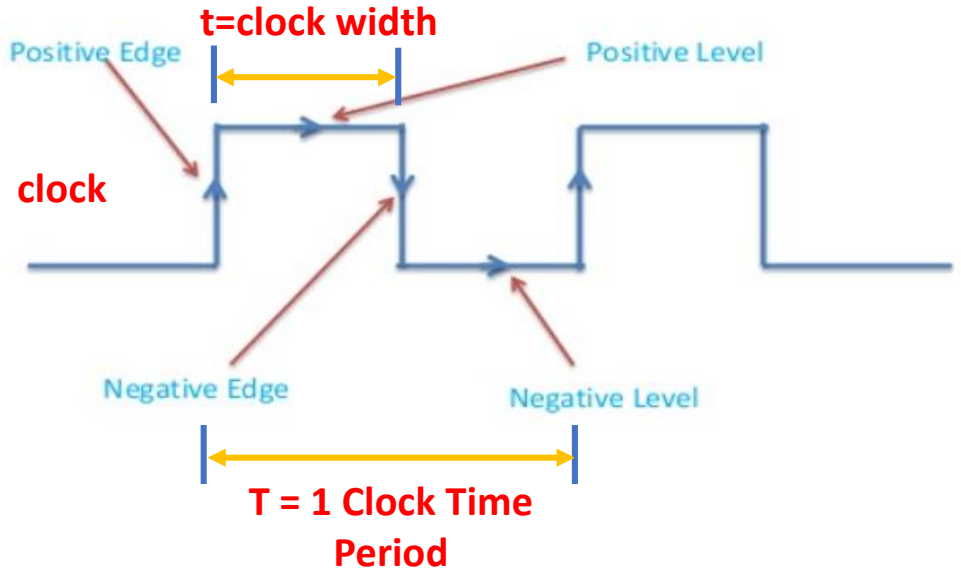
Clock

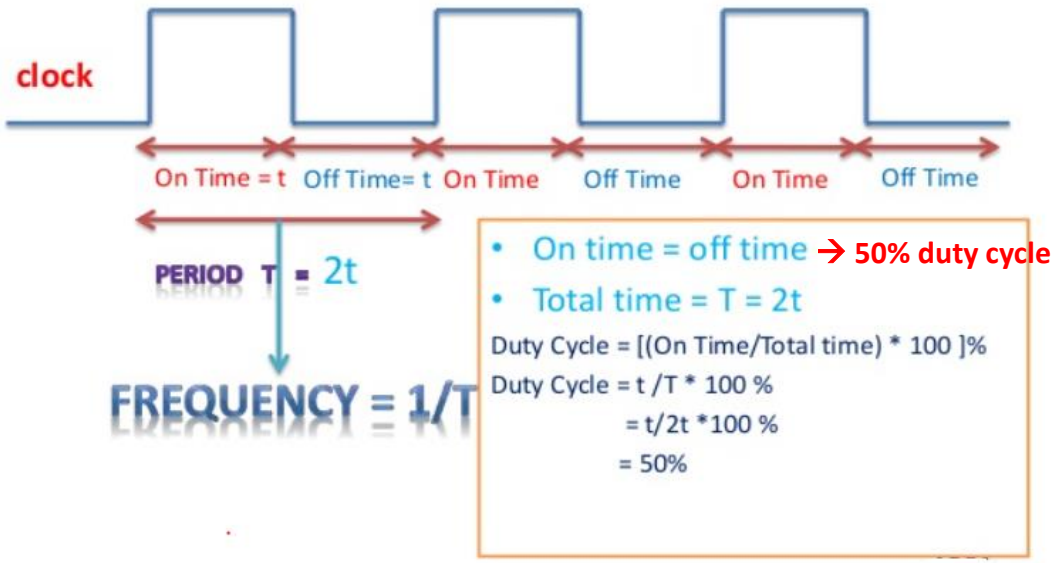
- **Clock is a signal which is repetitive in nature after some period of time**
  - In digital circuits, clock is a smallest unit of time when processing happens and sometimes it is also called as a “cycle”
- **Modern digital systems have multiple clocks where each of the core clock vibrates at a specific frequency**
  - Clock frequency is typically measured in MHz or GHz
- **Typically, in digital systems multiple clocks with desired frequency are generated using frequency divider from a single reference clock**



Clock Parameter Definitions

- **Positive edge =** 0 to 1 transition (also know as **Posedge or Rising Edge**)
- **Negative edge =** 1 to 0 transition (also know as **Negedge or Falling edge**)
- **Positive Level =** Positive edge to Negative edge (also know as **ACTIVE HIGH Level**)
- **Negative Level =** Negative edge to Positive edge (also know as **ACTIVE LOW Level**)
- **1 Clock Time Period (T) =** Time between successive transitions in the same direction
  - Positive edge to Positive edge OR Negative edge to Negative edge
- **Clock Width (t) =** Time during which the value of the clock signal is HIGH or LOW
- **Frequency =** 1 / Clock Time Period = **1 / T**
- **Duty Cycle =** Ratio of clock on time to clock off time (or Ratio of clock width **'t'** to clock period **'T'**)



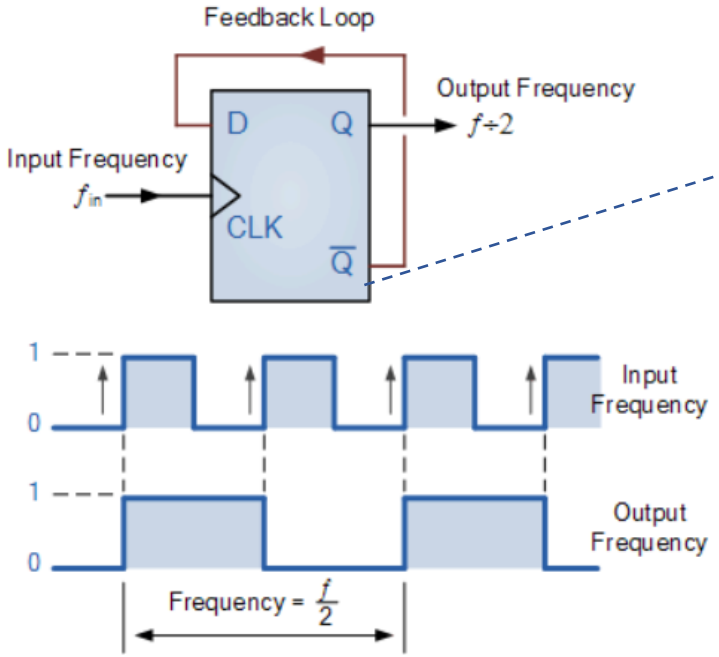


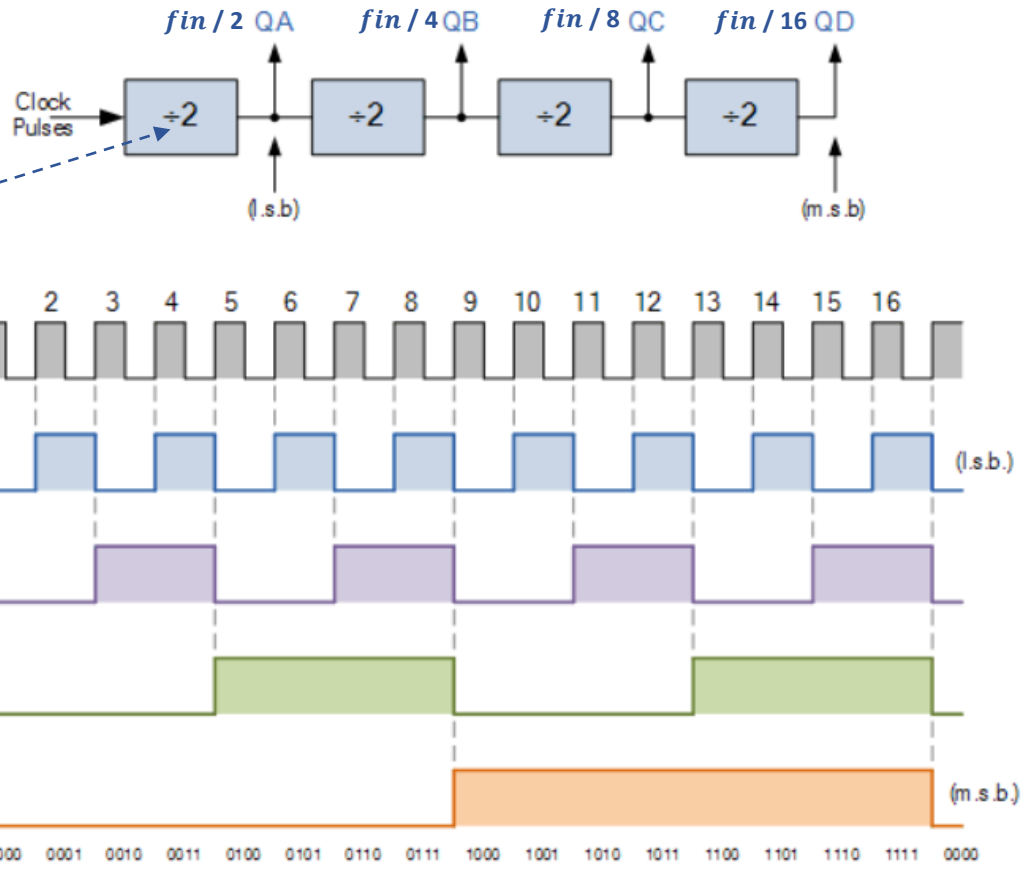
What is a Frequency Divider ?

- **A frequency divider, also called a clock divider, is a circuit that takes an input signal of a frequency (fin), and generates an output signal of a scaled down frequency ($fout$)**

$$fout = fin \therefore n, \qquad where\ n\ is\ an\ integer$$

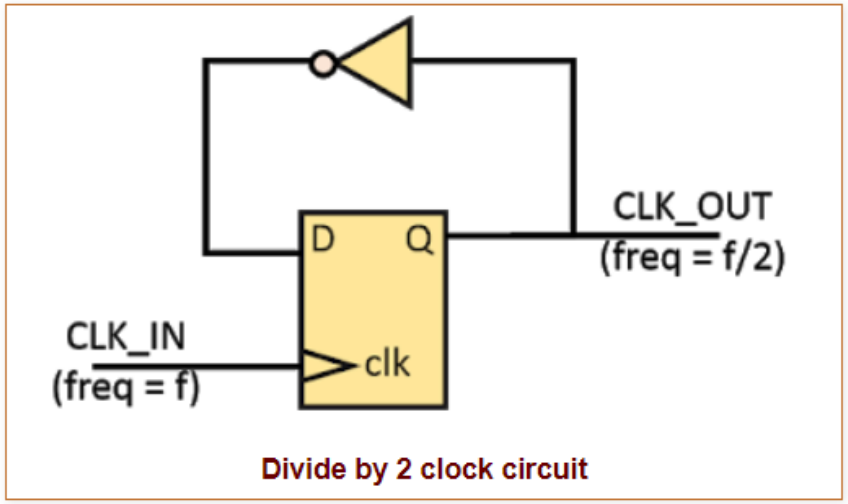
- **For power of 2 integer division toggle mode flip-flops are used in a chain as a divide by two counter.**
  - One flip-flop will divide the clock, f_{IN} by 2, two flip-flops will divide f_{IN} by 4 (and so on).
  - This is one of benefit of using toggle flip-flops for frequency division s that the output at any point has an exact 50% duty cycle





CLock Divide by 2

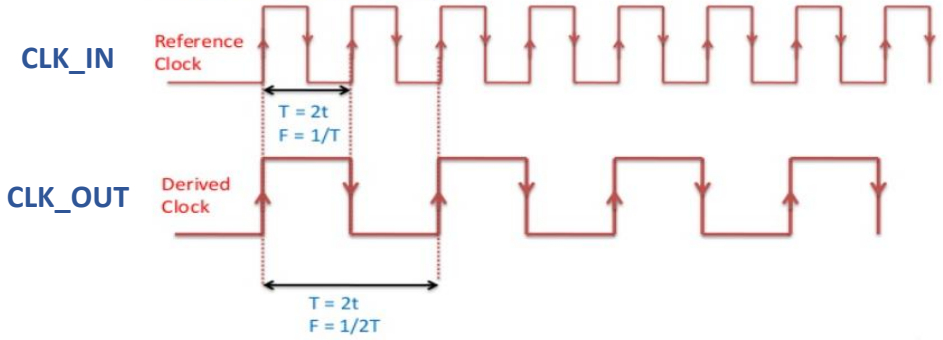
- **Clock divide by 2 can be developed using single D-FlipFlop with inverted output of flipfop connected to input 'D' of flipflop**
  - Reference clock (CLK_IN) is connected to flipflop clock pin
  - Flipflop output Q produces derived clock (CLK_OUT) which is 1/2 time of CLK_IN time period



Divide by 2 clock circuit

```
module clock_divide_by_2(
  input   CLK_IN, reset,
  output logic CLK_OUT);

always_ff@(posedge CLK_IN, posedge reset)
if (reset)
      CLK_OUT <= 0;
  else
      CLK_OUT <= ~CLK_OUT;
endmodule: clock_divide_by_4
```

- Freq divide By 2^N
- N=1 => Divide By 2



Clock Divide by 4

```
module clock_divide_by_4 #(parameter N = 4)(
 input   clkin, reset,
 output logic clkout);

 logic [$clog2(N)-1:0] cnt_value;
 always_ff@(posedge clkin, posedge reset)
  begin
   if (reset) begin
     cnt_value <= 0;
     clkout <= 0;
   end
   else if(cnt_value == $clog2(N)-1) begin
     cnt_value <= 0;
     clkout <= ~clkout;
   end
   else
     cnt_value <= cnt_value + 1;
  end
endmodule: clock_divide_by_4
```

For N=2, after cnt_value of 0, 1 clk_out value will be flipped from previous value

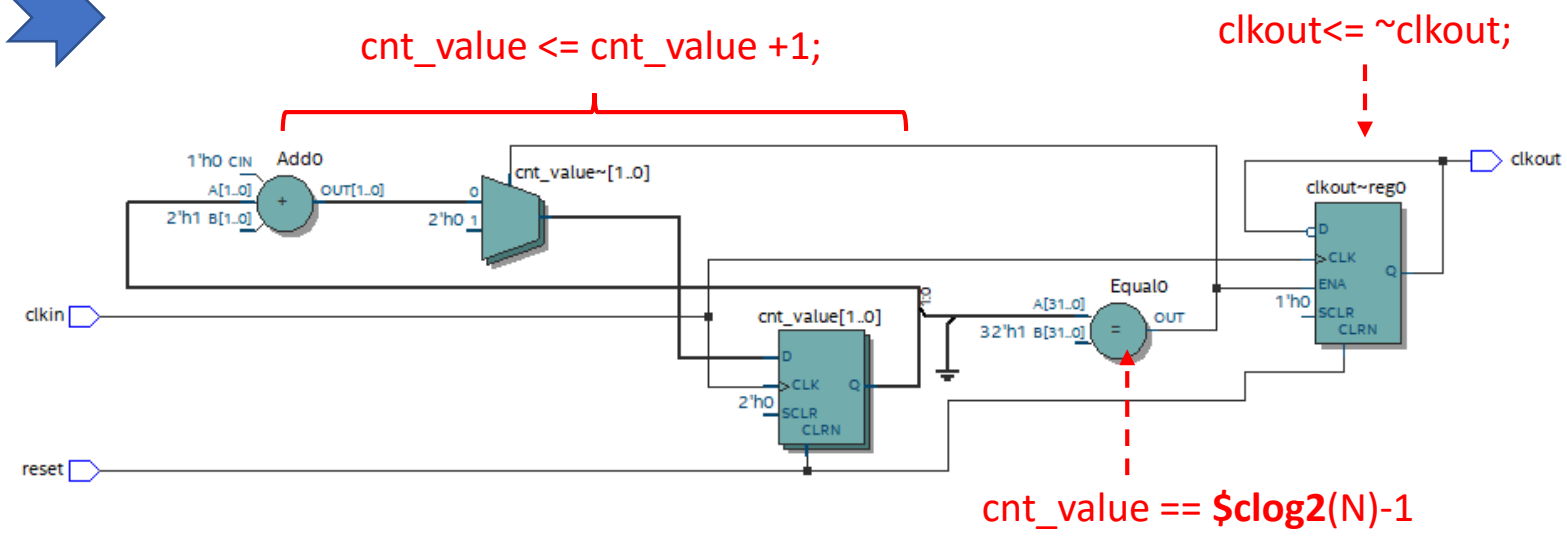
$clog2 function returns the ceiling of the logarithm to the base 2.

Example :

for N=4, $clog2(4) will return 2

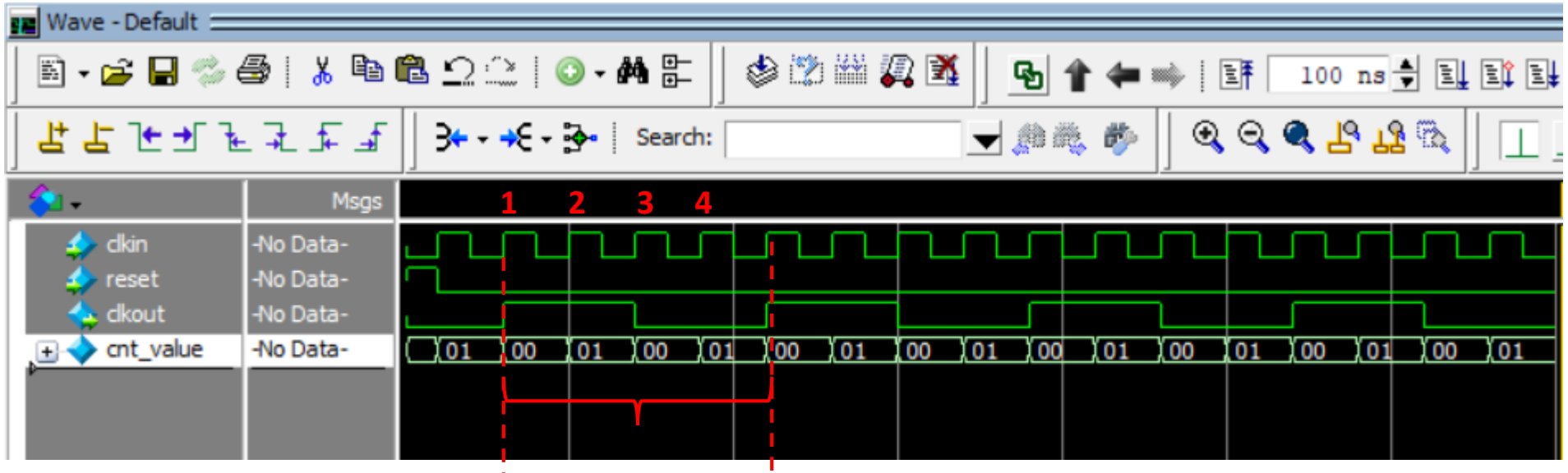
for N=6, $clog2(6) will return 3

For N=8, $clog2(8) will return 3



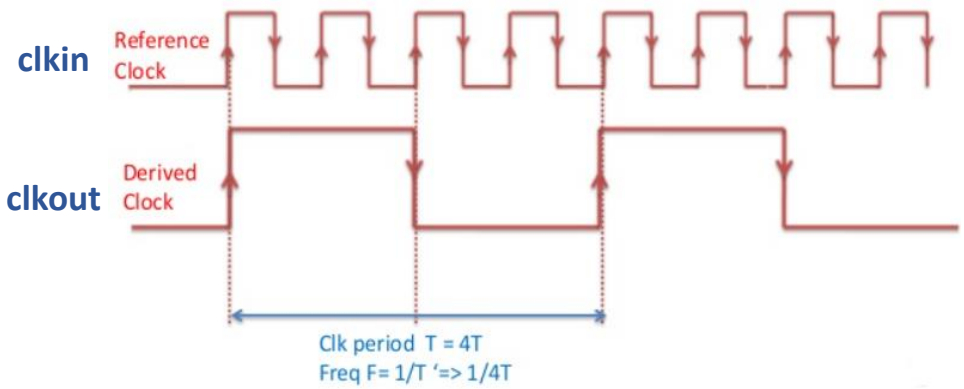
Question : If parameter N value is changed in SystemVerilog code to value 8, would above mentioned circuit work as divide by 8 ?

Clock Divide by 4 Example

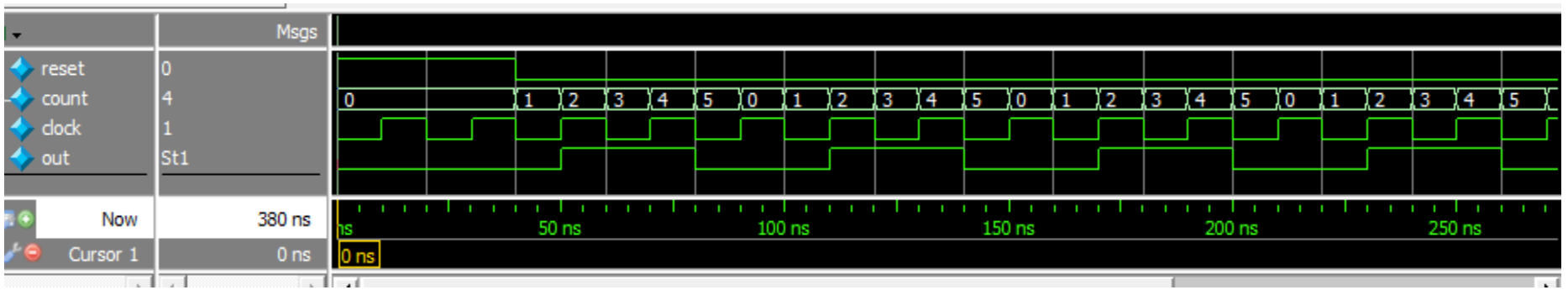


clkout is 4 times slower than clkin

- Freq divide By 2^N
- N=2 => Divide By 4



Let’s Design Clock Divide by 3



Note: Over a count of 6, we have three full clock cycles and one full out cycle.

Clock Frequency Divide by 3

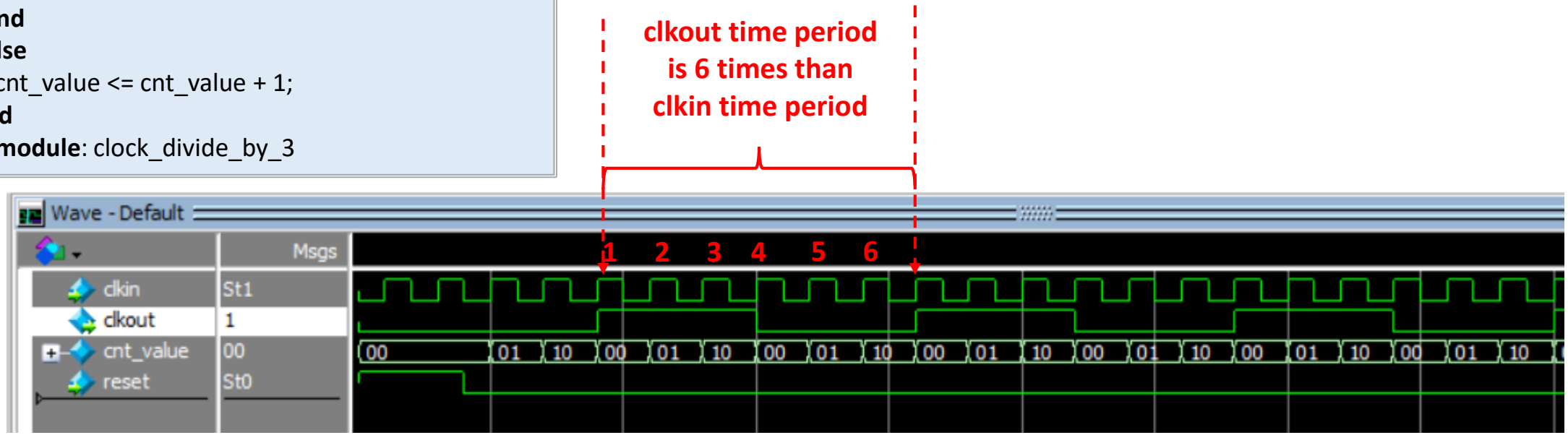
❑ Can clock divide by 4 implementation code re-used to create clock divide by 3 logic ?

```
module clock_divide_by_3(
 input  logic clkin, reset,
 output logic clkout);
logic [1:0] cnt_value;
always_ff@(posedge clkin or posedge reset)
  begin
   if (reset == 1) begin
     cnt_value <= 0;
     clkout <= 0;
   end
   else if(cnt_value == 2) begin
     cnt_value <= 0;
     clkout <= ~clkout;
   end
   else
     cnt_value <= cnt_value + 1;
  end
endmodule: clock_divide_by_3
```

Would changing cnt_value == 2 (i.e. counting 0,1,2) result in clock divide by 3 ?

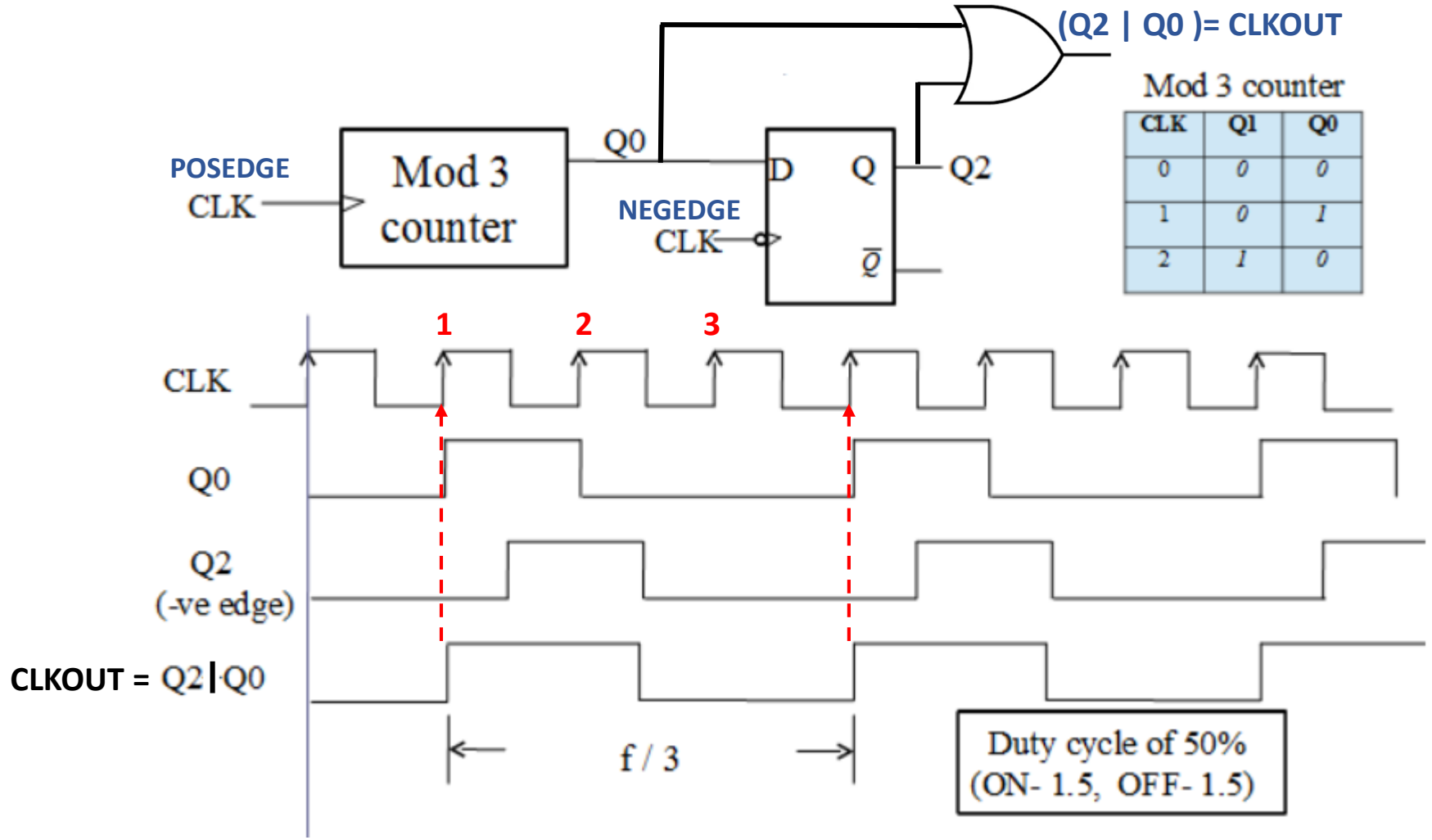
Answer : No !

It will behave as clock divide by 6 instead of 3 since cnt_value is incremented at every posedge of clkin, so clkout will be '1' for 3 clkin cycles and '0' for 3 clkin cycles.
So 1 clkout == 6 clkin. See below



Clock Divide by 3 (F/3) – Implementation 1

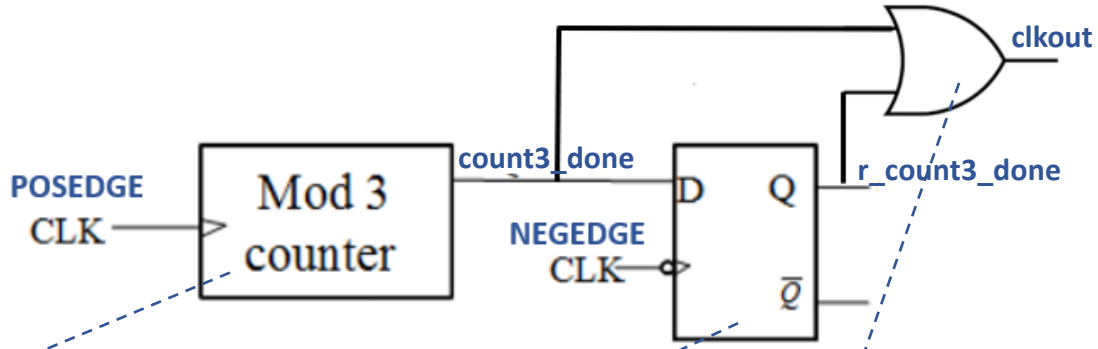
❑ Clock Frequency divide by 3 using Modulo-3 counter



Clock Divide by 3 (Implementation-1)

```
module clock_divide_by_3(
 input clkin, reset,
 output logic clkout);

 logic [1:0] cnt_value;
 logic count3_done, r_count3_done;

 always_ff@(posedge clkin, posedge reset) begin
  if (reset) begin
    cnt_value <= 0;
  end
  else begin
   if(cnt_value == 2) begin
      .....
      .....
   end
   else begin
     .....
     .....
   end
  end
 end

 …continued
```

Implement Modulo-3 counter which does binary count of 0,1,2 and once count value reaches 2, then set count3_done to = 1 and count value is reset to '0'



Implement negedge clk triggered flipflop to register output of modulo-3 counter "count3_done"

```
always_ff@(<add sensitivity list, remember negegde clk>) begin
  if (reset) begin
    .....
  end
  else begin
    .....
  end
end
  assign clkout = (count3_done | r_count3_done);
endmodule: clock_divide_by_3
```

Clock Divide by 3 (Implementation-2)

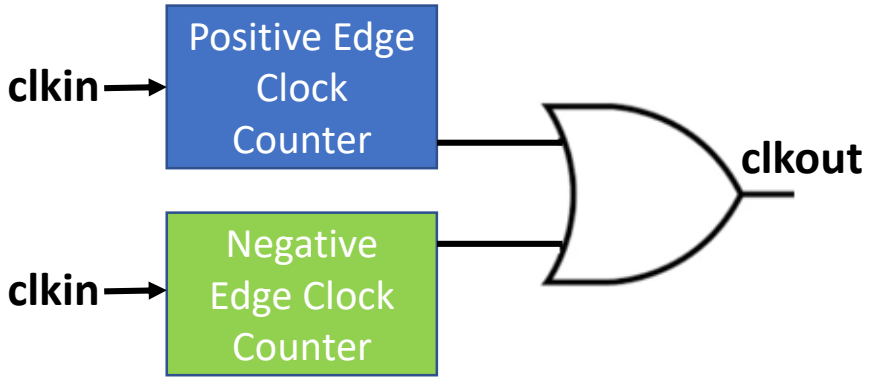
❑ Alternate implementation using two counters ?

- One counter for counting posedge of clkin and other counting negedge of clkin
- Whenever either counter reaches a count of '2' (i.e. counts 0,1,2 three edges then generate clkout=1

```
module clock_divide_by_3 (
 input  clkin, reset,
 output logic clkout);

 // local counter variable
 logic [1:0] posedge_count, negedge_count;

 // posedge clock counter
 always_ff@(posedge clkin) begin
  .....
  .....
 end

 // negedge clock counter
 always_ff@(negedge clkin) begin
  .....
  .....
 end

 // generate divide by 3 clock
 assign clkout = ((posedge_count == 2) | (negedge_count == 2));
endmodule: clock_divide_by_3
```

Add Reset condition
In Non-reset condition
increment posedge_count ?

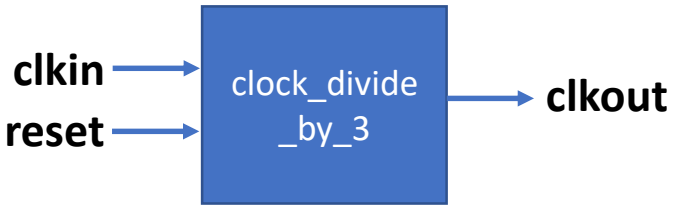
Add Reset condition
In Non-reset condition
increment negedge_count ?



Homework Assignment

- **Develop SystemVerilog RTL model for Clock Divide By 2N+1 (odd factor)**
  - Synthesize clock divide by 7 design and run simulation using testbench provided
  - Review synthesis results (resource usage and RTL netlist/schematic)
  - Review input and output signals in simulation waveform.
  - Assume below mentioned primary port names and SystemVerilog RTL module name **clock_divide_by_3.**

- **Primary Ports for carry_lookahead_adder module**
  - Input clkin, reset (Asynchronous active high reset)
  - Output clkout



- **Report should include :**
  - SystemVerilog design
  - Synthesis resource usage and schematic generated from RTL netlist viewer
  - Simulation snapshot and explain simulation result to confirm RTL model developed works as clock divide by 7